Republic of the Philippines
CAVITE STATE UNIVERSITY
Don Severino de las Alas Campus
Indang, Cavite

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY
Department of Information Technology

Name: _____ Student No.: _____
Course / Yr. / Sec.: _____ Date: _____

Activity #3 – A Simple Card Game

Objectives:

1. Each group should be able to create a working program that utilizes the concepts of object-oriented programming;
2. Each group must be able to present their working program.
3. Each group should be able to add another functionality to the proposed program.

Problem statement:

- Your group is developing a simple battle engine for a digital collectible card game. In this game, players use a limited resource called "Mana" to play the cards from their hand. Cards can either be Magic Type, Physical Type, or Pure Type of damage. Your task is to build the class structure that manages the player's hand, enforces mana costs, and calculates the total damage dealt in a single turn.

The Context:

- There are three distinct types of cards in this game:
  - **Physical Cards** – Standard attacks that cost mana and deal damage;
  - **Magic Cards** – Utility Spells that either can deal damage, heal the player, or restore mana;
  - **Pure Cards** – High risk, high reward attacks that deals a massive damage but drains the player's own HP during the attack.
- There is only one Generic **Boss** in this game, and the idea to finish the game is:
  - To defeat the Boss using the limited amount of **Mana**; and
  - The player must be able to stay alive until the end of its turn when **Mana** is completely burned.

The Programming Requirements: Below are the list of **classes** and its **attributes/methods** to be able to create the program.

1. **Player** and **Enemy Class**
   - **Player Class**
     - **Attributes** (Private) – `int hp`, `int mana`
     - **Methods: Constructor** to initialize both attributes, **getter** methods, and the additional methods below:
       1. `void takeDamage(int amount)`, void heal(int amount), `void spendMana(int amount)`, and `void restoreMana(int amount)`.
   - **Enemy Class**
     - **Attributes** (Private) – String name, int hp
     - **Methods: Constructor** to initialize both attributes, **getter** methods, and the additional methods below:
       1. `void takeDamage(int amount)`, and `boolean isDead()`.
     - **Additional Notes:** The enemy health should not drop below zero (0).
2. **Card Class** (The Abstract Class)
   - **Attributes** (Private) – `String name`, `int manaCost`
   - **Methods: Constructor** to initialize both attributes, **getter** methods, and additional methods below:
     - `void setManaCost(int cost)`, `abstract void play(Player player, Enemy boss)`, and `abstract void printDetails()`
     - **Additional Notes:** It should not allow a negative mana cost, the method `play()` will execute the card's specific logic against the player and enemy, and `printDetails()` display's the card stats.

3. **Physical Card** (Child Class of Card)
    - **Attributes** (Private): `int damage`
    - **Methods:** Constructor to initialize `damage`, overrides all abstract methods from `Card`.
    - **Additional Notes:** `play()` method always check if the user mana is sufficient. If not, it should prompt *"Not enough Mana!"* and if sufficient, it deducts mana, deals damage, and print a success message showing the damage dealt. The `printDetails()` prints the `name`, `cost`, and `damage` of the card.
4. **Magic Card** (Child Class of Card)
    - **Attributes** (Private): `int damage`, `int healAmount`, `int manaRestore`
    - **Methods:** Constructor to initialize the attributes, overrides all abstract methods from `Card`.
    - **Additional Notes:** `play()` method always check if the `player` mana is sufficient (and will deduct if sufficient), it deals `damage`, `heals` the `player`, and `restoreMana` to the `player`. Must print the applied effects. The printDetails() also prints the `name`, `cost`, `damage`, `heal`, and `mana restore` values.
5. **Pure Card** (Child Class of Card)
    - **Attributes** (Private): `int damage`, `int selfDamage`
    - **Methods:** Constructor to initialize the attributes, overrides all abstract methods from `Card`.
    - **Additional Notes:** `play()` method always check if the `player` mana is sufficient (and will deduct if sufficient), deals `damage` to the `Boss`, and deals `selfDamage` to the `player`. Prints the damage and the self-inflicted damage. `printDetails()` also prints the `name`, `cost`, `damage`, and `selfDamage`.
6. **Deck** (The Card Manager Class)
    - **Attributes:** `ArrayList<Card> cardPool`, `Random randomGenerator`
    - **Methods:** Constructor that initializes the `ArrayList` and `Random` instance, then immediately populates the list with the 13 pre-defined cards (which is listed below)
        - `Card drawRandomCard()` – this method generates a random index based on the size of the `cardPool` and returns the card at that index.
7. **The Manager Class (BattleEngine Class)**
    - **Attributes:** ArrayList<Card> hand
    - **Methods:**
        - **Constructor** that initializes the list
        - `void drawHand(Deck deck, int numberOfCards)` – uses a loop to call `deck.drawRandomCard()` and adds the returned cards to the hand.
        - `void executeTurn(Player player, Enemy boss)` – iterates through the random hand, printing card details and attempting to play them until the hand is empty or the boss is dead.

The 13 Cards:

- **Physical Cards (**`name, manaCost, damage`**)**
    - `PhysicalCard("Quick Attack", 1, 10);`
    - `PhysicalCard("Shuriken Toss", 2, 20);`
    - `PhysicalCard("Double Edge", 3, 35);`
    - `PhysicalCard("Dragon Tail", 1, 12);`
    - `PhysicalCard("Primal Roar", 2, 15);`
- **Magic Cards (**`name, manaCost, damage, healAmount, manaRestore`**)**
    - `MagicCard("Lucent Beam", 2, 5, 0, 0);`
    - `MagicCard("Mana Drain", 0, 3, 0, 3);`
    - `MagicCard("Shallow Grave", 5, 0, 30, 0);`
    - `MagicCard("Finger of Death", 10, 25, 0, 0);`
    - `MagicCard("Mystic Snake", 2, 15, 15, 0);`
- **Pure Cards (**`name, manaCost, damage, selfDamage`**)**
    - `PureCard("Wild Axes", 5, 15, 0);`
    - `PureCard("Consume", 5, 40, 25);`
    - `PureCard("Purifying Flames", 3, 50, 35);`

The Expected Output:

```
run:
--- Boss Battle Begins ---
Player | HP: 50 | Mana: 5
Card: Mana Drain
Mana Cost: 0 Mana
Damage: 3
Heal Amount: 0
Mana Restored: 3
-------------------------------------
Card: Primal Roar
Mana Cost: 2 Mana
Damage: 15
-------------------------------------
Player casts Primal Roar! Dealt 15 damage to Badong the Great.
Card: Shuriken Toss
Mana Cost: 2 Mana
Damage: 20
-------------------------------------
Player casts Shuriken Toss! Dealt 20 damage to Badong the Great.
Card: Purifying Flames
Mana Cost: 3 Mana
Damage: 50
Self-Damage: 35
-------------------------------------
Player used Purifying Flames! Dealt 50 pure damage, but took 35 recoil damage.

--- Battle Finished! ---
Player Final Status -> HP: 15 | Mana: 1
Boss Final Status -> HP: 12
BUILD SUCCESSFUL (total time: 0 seconds)
```

Main Class:

```
public static void main(String[] args) {
    Player hero = new Player(50, 5);
    Enemy boss = new Enemy("Badong the Great", 100);

    Deck masterDeck = new Deck();
    Engine engine = new Engine();

    engine.drawHand(masterDeck, 4);

    engine.executeTurn(hero, boss);
}
```